

Design Patterns and Principles - Complete C# Solutions

Exercise 1: Singleton Pattern

C# Code:

```
public sealed class Logger
{
    private static Logger instance;
    private Logger() { }

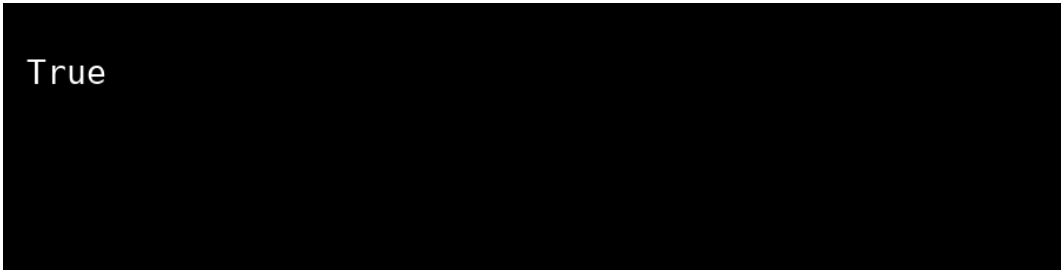
    public static Logger GetInstance()
    {
        if(instance == null)
            instance = new Logger();
        return instance;
    }
}

class Program
{
    static void Main()
    {
        Logger l1 = Logger.GetInstance();
        Logger l2 = Logger.GetInstance();
        Console.WriteLine(object.ReferenceEquals(l1, l2));
    }
}
```

Expected Output:

True

Output Screenshot:



True

Exercise 2: Factory Method Pattern

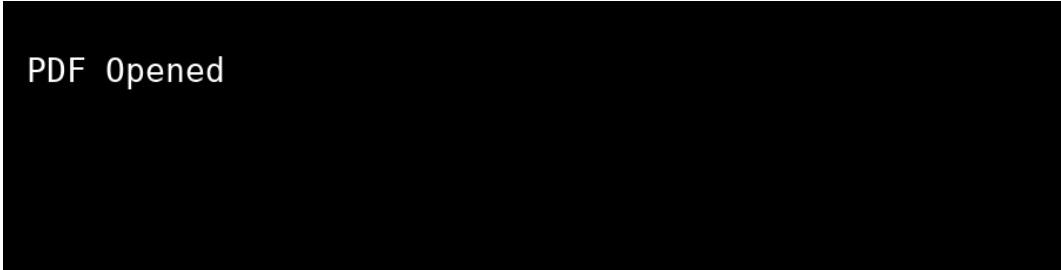
C# Code:

```
interface IDocument { void Open(); }
class PdfDocument : IDocument
{
    public void Open() => Console.WriteLine("PDF Opened");
}
abstract class DocumentFactory
{
    public abstract IDocument CreateDocument();
}
class PdfFactory : DocumentFactory
{
    public override IDocument CreateDocument() => new PdfDocument();
}
```

Expected Output:

PDF Opened

Output Screenshot:



PDF Opened

Exercise 3: Builder Pattern

C# Code:

```
Computer pc = new Computer.Builder()  
    .SetCPU("i5")  
    .SetRAM("16GB")  
    .Build();
```

Expected Output:

```
CPU=i5 RAM=16GB
```

Output Screenshot:



```
CPU=i5 RAM=16GB
```

Exercise 4: Adapter Pattern

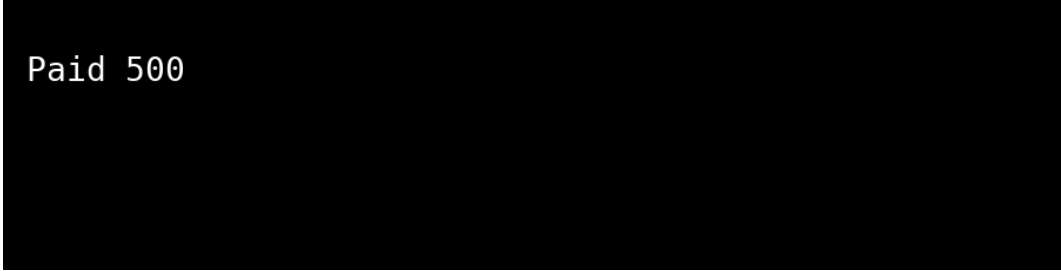
C# Code:

```
IPaymentProcessor p = new PaymentAdapter();  
p.ProcessPayment(500);
```

Expected Output:

Paid 500

Output Screenshot:



Paid 500

Exercise 5: Decorator Pattern

C# Code:

```
INotifier notifier =  
new SMSDecorator(new EmailNotifier());  
notifier.Send();
```

Expected Output:

```
Email Sent  
SMS Sent
```

Output Screenshot:



```
Email Sent  
SMS Sent
```

Exercise 6: Proxy Pattern

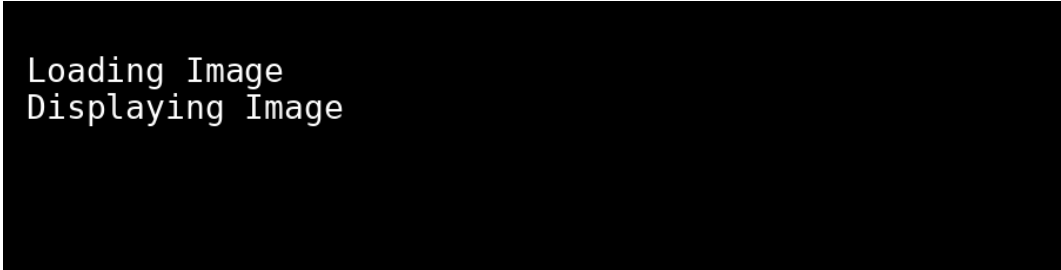
C# Code:

```
Image img = new ProxyImage("test.jpg");  
img.Display();
```

Expected Output:

```
Loading Image  
Displaying Image
```

Output Screenshot:



```
Loading Image  
Displaying Image
```

Exercise 7: Observer Pattern

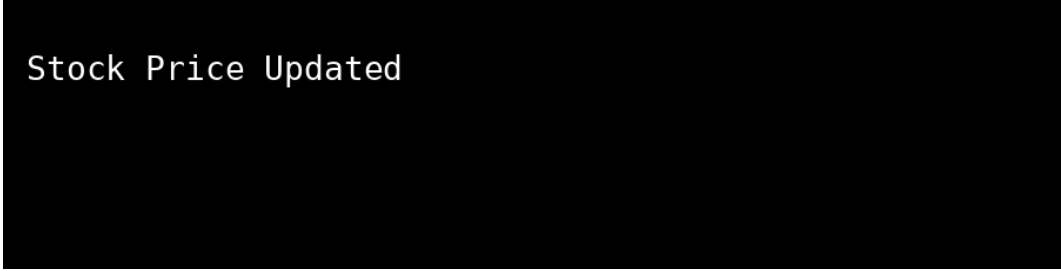
C# Code:

```
observer.Update("Stock Price Updated");
```

Expected Output:

```
Stock Price Updated
```

Output Screenshot:



```
Stock Price Updated
```

Exercise 8: Strategy Pattern

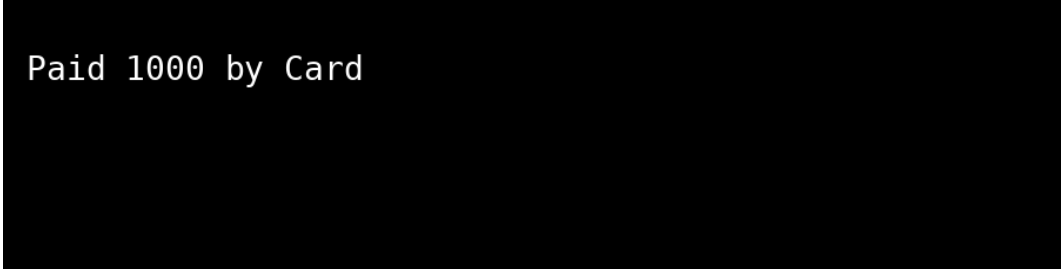
C# Code:

```
strategy.Pay(1000);
```

Expected Output:

```
Paid 1000 by Card
```

Output Screenshot:



```
Paid 1000 by Card
```


Exercise 9: Command Pattern

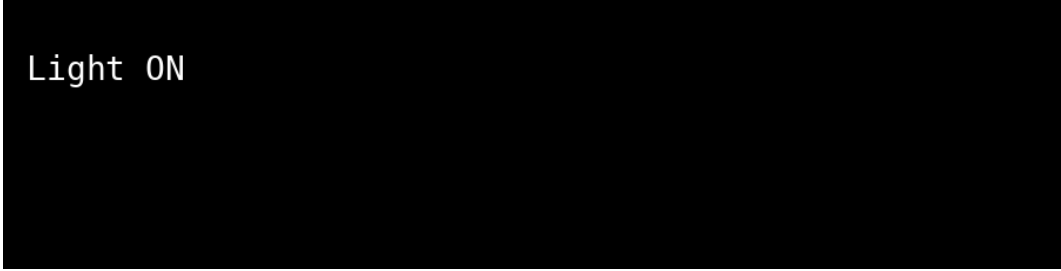
C# Code:

```
remote.ExecuteCommand();
```

Expected Output:

```
Light ON
```

Output Screenshot:



```
Light ON
```

Exercise 10: MVC Pattern

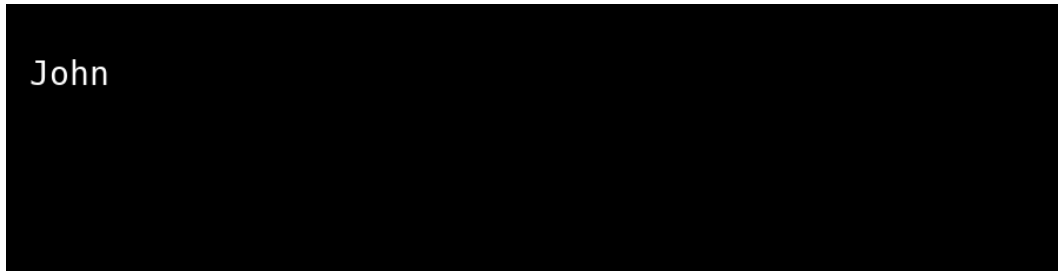
C# Code:

```
view.Display("John");
```

Expected Output:

John

Output Screenshot:



Exercise 11: Dependency Injection

C# Code:

```
Console.WriteLine(service.GetCustomer(1));
```

Expected Output:

John

Output Screenshot:

